	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Rene Adrián Davila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 7

Integrante(s): 323001315, 426057857, 323008754, 426058665, 323067267

No. de lista o brigada: 5

Semestre: 2027-1

Fecha de entrega: 16 de Octubre

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **2**
1.1. Planteamiento del problema. 2
1.2. Motivación. 2
1.3. Objetivos. 2

2. Marco Teórico **3**
2.1. Herencia y la Clase Object 3
2.2. Relaciones IS-A y HAS-A 3
2.3. Sobrecarga y Sobrescritura de Métodos 3

3. Desarrollo **3**
3.1. Ejercicios de Práctica 3

4. Resultados **5**
4.1. Ejercicio 2 5
4.2. Ejercicio 1 6

5. Conclusiones **7**

1. Introducción

1.1. Planteamiento del problema.

El problema a resolver consiste en construir una jerarquía de herencia lo más profunda posible para representar figuras geométricas, partiendo de la superclase `Cuadrilatero` y derivando de ella las clases `Trapezoide`, `Paralelogramo`, `Rectangulo` y `Cuadrado`. Para ello, las variables de instancia privadas de la superclase deben ser los pares de coordenadas (x, y) de sus cuatro vértices, requiriendo la creación de una clase `Punto` que represente dichas coordenadas. Además, la aplicación debe ser capaz de crear instancias de cada clase y calcular su respectiva área (a excepción de la clase base), adaptando la fórmula general del trapecio $A = \frac{(B+b)h}{2}$ a los casos específicos de cada figura, como $A = b \cdot h$ o $A = b^2$. Finalmente, el diseño debe quedar documentado mediante diagramas UML.

1.2. Motivación.

En el ámbito del desarrollo de software y la ingeniería en computación, es necesario dar solución a este problema para consolidar el uso de la herencia, un pilar que promueve la reutilización de código al permitir que nuevos objetos asuman las propiedades de objetos existentes. Resolver este ejercicio ayuda a comprender de forma práctica cómo aplicar y diferenciar relaciones estructurales clave, como la relación *IS-A* (es un) entre los distintos niveles jerárquicos de las figuras, y la relación *HAS-A* (tiene un) al componer el cuadrilátero a partir de instancias de puntos.

1.3. Objetivos.

Lo que se espera obtener al darle solución al problema es:

- Implementar los conceptos de herencia en un lenguaje de programación orientado a objetos para generar una jerarquía de clases funcional.
- Crear una jerarquía lo más profunda posible utilizando `Cuadrilatero` como superclase.
- Emplear la clase `Punto` para gestionar la encapsulación de las coordenadas de cada figura geométrica.
- Especificar correctamente las variables de instancia y los métodos de cada clase, logrando instanciar objetos y calcular sus áreas derivadas de la fórmula base del trapecio.
- Representar la estructura lógica y la implementación del sistema elaborando los diagramas UML correspondientes.

2. Marco Teórico

2.1. Herencia y la Clase Object

La herencia es un mecanismo fundamental de la Programación Orientada a Objetos que permite crear nuevos objetos a partir de otros preexistentes, adquiriendo sus propiedades y métodos visibles, además de poder agregar funcionalidades propias. A la clase que sirve como base se le denomina superclase, mientras que la que hereda se conoce como subclase o clase derivada. Cabe destacar que toda clase creada por el desarrollador hereda implícitamente de la clase `Object`. Este pilar se utiliza principalmente para promover la reutilización de código y habilitar el uso del polimorfismo. Su sintaxis en Java se define mediante la palabra reservada `extends`.[1]

2.2. Relaciones IS-A y HAS-A

El modelado de jerarquías de objetos se fundamenta en relaciones estructurales. La relación IS-A (Es un) se basa en la herencia y permite afirmar que un objeto pertenece a un tipo o clase en específico. Por ejemplo, al aplicar la herencia, se puede establecer lógicamente que un Cuadrado es un Rectángulo. Por otro lado, la relación HAS-A (Tiene un) es una asociación de composición basada en el uso más que en la herencia. Un ejemplo de esto ocurre cuando una clase como `Cuadrilatero` no hereda de un punto, sino que contiene vértices compuestos por instancias de la clase `Punto`.[2]

2.3. Sobrecarga y Sobrescritura de Métodos

Para calcular áreas específicas y manipular el comportamiento de los objetos, se aplican técnicas fundamentales. La sobrescritura (overriding) consiste en redefinir el comportamiento de un método heredado en una subclase, como se observa al adaptar la fórmula matemática en el método `calcularArea()` para cada figura o al redefinir `toString()` y `equals()` heredados de `Object`. Por su parte, la sobrecarga (overloading) permite declarar múltiples métodos o constructores con el mismo nombre dentro de una clase, diferenciándose exclusivamente por la cantidad o tipo de sus parámetros, brindando versatilidad en la inicialización y operación de los objetos.[3]

3. Desarrollo

3.1. Ejercicios de Práctica

Ejercicio 1

1. **Consulta a la IA.** Se consulta la pregunta acerca de por qué no es bueno el uso del encapsulamiento `protected` en herencia a la IA generativa y se lee la respuesta completa.

2. **Análisis de la información.** Se revisan los argumentos que la IA da sobre el uso de `protected` y se identifican las razones principales.

3. **Identificación de alternativas.** Se anotan las alternativas que la IA propone y los motivos que da para recomendarlas.

4. **Verificación de la información.** Se contrasta lo obtenido con la documentación oficial de Oracle sobre modificadores de acceso en Java.

5. **Registro de la consulta.** Se anotan en el reporte la pregunta formulada, la IA utilizada y un resumen de la respuesta.

6. **Debate entre integrantes.** Se reúnen las respuestas de los integrantes y se contrastan para identificar coincidencias y diferencias.

7. **Conclusión.** Se resumen los puntos en común y se indica la postura a la que llegó el equipo.

Ejercicio 2

1. **Inicio del trabajo.** Se abre la terminal de VS Code y se crea una carpeta para la práctica, donde se guardarán los archivos `.java`.

2. **Definición de la jerarquía.** Cada figura es un caso particular de la anterior, así que las clases quedan en este orden: `Cuadrilatero` → `Trapezoide` → `Paralelogramo` → `Rectangulo` → `Cuadrado`. Los vértices se representan con la clase `Punto`.

3. **Clase Punto.** Se crea con los atributos privados x e y, el constructor y los métodos `getX()` y `getY()`.

4. **Clase Cuadrilatero.** Es la superclase. Guarda cuatro vértices privados de tipo `Punto` y tiene un método `getVertice` por cada uno. No tiene método de área.

5. **Clase Trapezoide.** Hereda de `Cuadrilatero` y calcula el área con $A = \frac{(B+b)\cdot h}{2}$.

6. **Clase Paralelogramo.** Hereda de `Trapezoide` y sobrescribe `calcularArea()`, porque cuando $B = b$ la fórmula es $A = b \cdot h$.

7. **Clase Rectangulo.** Hereda de `Paralelogramo`, usa la misma fórmula y agrega `obtenerArea()`, que llama a `calcularArea()`.

8. **Clase Cuadrado.** Hereda de `Rectangulo` and sobrescribe `calcularArea()`, porque cuando $b = h$ la fórmula es $A = b^2$.

9. **Programa principal.** En `Main` se crea un objeto de cada clase con puntos de prueba y se imprime el área de todos, menos la del `Cuadrilatero`.

10. **Cierre del trabajo.** Se comprueba que las áreas obtenidas coincidan con las calculadas a mano.

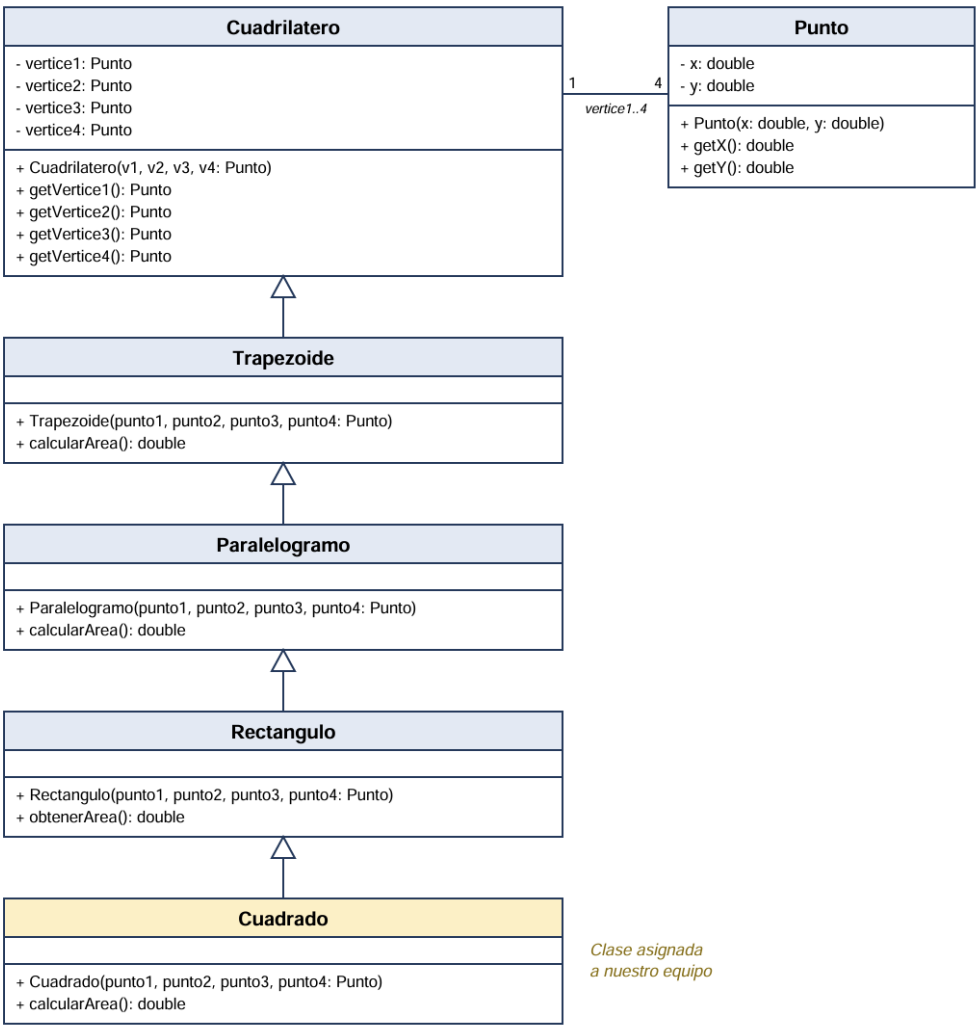


Figura 1: Diagrama de clases ejercicio 2

4. Resultados

4.1. Ejercicio 2

Cuadriláteros

A continuación, se presenta la captura de pantalla con la salida en consola tras la ejecución del programa principal (`Main.java`).

El programa instancia objetos de cada una de las figuras de la jerarquía (Trapezoide, Paralelogramo, Rectángulo y Cuadrado), asignando coordenadas específicas a sus vértices mediante la clase `Punto`. Posteriormente, el sistema calcula y muestra las áreas correspondientes para evidenciar el correcto funcionamiento de la sobrescritura y sobrecarga de métodos. Finalmente, la ejecución valida las relaciones estructurales de herencia (IS-A) mediante el operador `instanceof`, la composición (HAS-A), y la

redefinición de los métodos heredados de la superclase `Object` como `toString()` y `equals()`.

```
PS C:\Users\acer\Downloads\Practica-7-Equipo-5-main> cd Practica-7-Equipo-5-main\mx\unam\fi\die\poo\g7\src
PS C:\Users\acer\Downloads\Practica-7-Equipo-5-main\Practica-7-Equipo-5-main\mx\unam\fi\die\poo\g7\src> javac mx\unam\
fi\die\poo\g7\practica07\*.java
PS C:\Users\acer\Downloads\Practica-7-Equipo-5-main\Practica-7-Equipo-5-main\mx\unam\fi\die\poo\g7\src> java mx.unam.f
i.die.poo.g7.practica07.Main
===== 1. Areas =====
Area del Trapezoide:     24.0
Area del Paralelogramo:  24.0
Area del Rectangulo:     24.0
Area del Cuadrado:       25.0

===== 2. Clase Object: toString() y equals() =====
Cuadrilatero [(0.0, 0.0), (4.0, 0.0), (5.0, 3.0), (1.0, 2.0)]
Cuadrado [(0.0, 0.0), (5.0, 0.0), (5.0, 5.0), (0.0, 5.0)]
a == b      : false
a.equals(b) : true

===== 3. Relacion IS-A (herencia) =====
cuadrado es Rectangulo:    true
cuadrado es Cuadrilatero:  true
trapezoide es Cuadrado:    false

===== 4. Relacion HAS-A (composicion) =====
Un Cuadrilatero TIENE 4 puntos. Vertice 1 = (0.0, 0.0)

===== 5. Sobrecarga (overloading) =====
new Punto()                      = (0.0, 0.0)
trapezoide.calcularArea()        = 24.0
trapezoide.calcularArea(8, 4, 4) = 24.0

===== 6. Sobrescritura (overriding) =====
Trapezoide figura = cuadrado; figura.calcularArea() = 25.0
PS C:\Users\acer\Downloads\Practica-7-Equipo-5-main\Practica-7-Equipo-5-main\mx\unam\fi\die\poo\g7\src>
```

Figura 2: Resultado ejercicio 2

4.2. Ejercicio 1

¿Porqué no es buena práctica la encapsulación protected en la Herencia?

El uso del modificador `protected` directamente en los atributos de una clase base se considera una mala práctica en la Programación Orientada a Objetos porque rompe la verdadera encapsulación y genera un diseño de software frágil a largo plazo.

Aunque conceptualmente parece tener sentido compartir datos con las clases hijas, en la práctica causa tres problemas principales en el diseño:

- **Pérdida de control del estado y las invariantes.** La encapsulación sirve para que una clase sea la única responsable de proteger la integridad de sus propios datos. Si un atributo es protegido, cualquier subclase puede modificar ese valor directamente y asignarle datos no válidos, saltándose las validaciones o reglas de negocio que la clase padre intentaba imponer.

- **Acoplamiento fuerte.** Este inconveniente, también conocido como el problema de la clase base frágil, ocurre porque las subclases se vuelven dependientes de la implementación interna exacta del padre. Si en el futuro es necesario cambiar

el nombre del atributo, su tipo de dato, o la manera en que se calcula internamente, inevitablemente se romperá la compatibilidad de todas las subclases que dependían de él, obligando a refactorizar en cascada.

- **Falsa restricción de visibilidad.** En entornos de desarrollo estructurados por paquetes, el modificador protegido suele otorgar acceso no solo a las subclases, sino a cualquier otra clase que se encuentre dentro del mismo directorio o paquete, exponiendo la información mucho más allá de la jerarquía de herencia estricta.

La regla general para mantener un diseño robusto establece que todos los atributos deben ser privados sin excepción. Si una subclase legítimamente necesita consultar o alterar el estado de su clase padre, la clase base debe exponer métodos protegidos o públicos (getters, setters o métodos de comportamiento) para hacerlo. De esta manera, las clases se comunican mediante un contrato seguro y la clase padre mantiene el control total sobre su lógica interna.

5. Conclusiones

A partir de los resultados, se evidencia que una jerarquía de herencia profunda permite estructurar soluciones eficientes y con alta reutilización de código. Delegar los vértices mediante composición (HAS-A) en la clase base `Cuadrilatero` y especializar las figuras derivando clases (IS-A), demuestra que una correcta abstracción simplifica significativamente el diseño de software.

Asimismo, la precisión en los cálculos resalta la utilidad del polimorfismo. La sobrescritura (overriding) del método `calcularArea()` permitió que cada figura resolviera su área de forma autónoma, mientras que la sobrecarga (overloading) brindó flexibilidad lógica. Además, redefinir los métodos implícitos de la clase `Object` (`toString()` y `equals()`) probó ser indispensable para la correcta representación y comparación de las instancias.

Finalmente, el análisis confirma que un diseño de clases robusto depende de una estricta encapsulación. Mantener el estado como `private`, en lugar de utilizar `protected` en la herencia, es vital para prevenir el acoplamiento fuerte y asegurar que se extiendan comportamientos sin comprometer la integridad interna de las clases.

Referencias

[1] J. A. Solano Gálvez, M. A. Nakayama C., y A. Velázquez Mena, *Manual de prácticas del Laboratorio de Programación orientada a objetos*, Versión 01. Ciudad de México, México: Facultad de Ingeniería, UNAM, 2017, pp. 112-126.

[2] P. Deitel y H. Deitel, *Cómo programar en Java*, 7.ª ed. Naucalpan de Juárez, México: Pearson Educación, 2008.

[3] D. J. Barnes y M. Kölling, *Programación Orientada a Objetos con Java*, 3.ª ed. Madrid, España: Pearson Educación, 2007.